University of Colombo
School of Computing

SCS 2312

Computational Models and Programming Language Concepts

Take-home Assignment

Parser Implementation

Student Information

Name:	R K K Jinendra
Index Number:	23000821

GitHub Repository:

https://github.com/Kalana2/parser.git

February 15, 2026

Contents

1 Context-Free Grammar (CFG)

1.1 Grammar Productions

$$
\begin{aligned}
\text{Program} &\rightarrow \text{StatementList} \\
\text{StatementList} &\rightarrow \text{Statement} \mid \text{Statement StatementList} \\
\text{Statement} &\rightarrow \text{Declaration} \mid \text{Assignment} \mid \text{Print} \\
\text{Declaration} &\rightarrow \texttt{int}\ \text{IDENTIFIER}\ \texttt{;} \\
&\quad \mid \texttt{int}\ \text{IDENTIFIER}\ \texttt{=}\ \text{Expression}\ \texttt{;} \\
\text{Assignment} &\rightarrow \text{IDENTIFIER}\ \texttt{=}\ \text{Expression}\ \texttt{;} \\
\text{Print} &\rightarrow \texttt{print (}\ \text{IDENTIFIER}\ \texttt{);} \\
\text{Expression} &\rightarrow \text{Term} \mid \text{Term}\ \texttt{+}\ \text{Expression} \\
\text{Term} &\rightarrow \text{NUMBER} \mid \text{IDENTIFIER}
\end{aligned}
$$

1.2 Terminal Symbols

- **Keywords:** `int`, `print`
- **Operators:** `+`, `=`
- **Symbols:** `(`, `)`, `;`
- **IDENTIFIER:** Variable names matching `[a-zA-Z_][a-zA-Z0-9_]*`
- **NUMBER:** Integer literals matching `[0-9]+`

1.3 Non-Terminal Symbols

- **Program:** The start symbol representing the entire program
- **StatementList:** A sequence of one or more statements
- **Statement:** A single statement (declaration, assignment, or print)
- **Declaration:** Variable declaration with optional initialization
- **Assignment:** Variable assignment
- **Print:** Print statement to display variable value
- **Expression:** Arithmetic expression with addition
- **Term:** Basic expression element (number or variable)

2 Implementation Overview

2.1 Tokenizer

The tokenizer scans the input file character-by-character and identifies tokens:

```
void tokenize(FILE *file) {
    // Reads characters and classifies them into:
    // - KEYWORD: Reserved words (int, print)
    // - IDENTIFIER: Variable names
    // - NUMBER: Integer literals
    // - OPERATOR: Arithmetic/assignment operators (+, =)
    // - SYMBOL: Delimiters ( ) ;
}
```

Listing 1: Tokenizer Function

Key Features:

- Ignores whitespace
- Identifies multi-character tokens
- Stores tokens in an array for parsing
- Detects unrecognized tokens

2.2 Token Recognition Functions

```
int isKeyword(char *str);     // Checks against keyword list
int isOperator(char *str);    // Checks against operator list
int isSymbol(char *str);      // Checks against symbol list
int isIdentifier(char *str); // Validates identifier format
int isNumber(char *str);      // Validates number format
```

Listing 2: Token Classification

2.3 Parser Structure

The parser uses recursive descent parsing following the CFG:

```
void parseProgram();          // Entry point: StatementList
void parseStatementList();    // Parses multiple statements
void parseStatement();        // Routes to specific statement type
void parseDeclaration();      // Handles: int x; or int x = expr;
void parseAssign();           // Handles: x = expr;
void parsePrint();            // Handles: print(x);
int parseExpression();        // Evaluates arithmetic expressions
```

Listing 3: Parser Functions

3 Error Detection

3.1 Syntax Errors

The parser detects syntax errors through the `expect()` function:

```
void expect(char *type, char *value) {
    // Checks if current token matches expected type and value
    // Exits with error message if mismatch occurs
}
```

Listing 4: Syntax Error Detection

Examples:

- Missing semicolon: `int x = 5` (missing `;`)
- Invalid token sequence: `int = 5;`
- Unexpected end of input

3.2 Semantic Errors

Semantic errors are detected during variable operations:

```
// 1. Variable redeclaration
void addVariable(char *name, int value, int isInitialized) {
    // Checks if variable already exists
    // Error: Variable 'x' already declared
}

// 2. Undefined variable
int lookupVariable(char *name) {
    // Searches for variable in symbol table
    // Error: Variable 'x' not found
}

// 3. Uninitialized variable
int lookupVariable(char *name) {
    // Checks if variable is initialized before use
    // Error: Variable 'x' is not initialized
}
```

Listing 5: Semantic Error Detection

Examples:

- Using undeclared variable: `print(y);` when `y` not declared
- Using uninitialized variable: `int x; print(x);`
- Redeclaring variable: `int x; int x;`

4 Symbol Table

The parser maintains a symbol table for variable management:

```
typedef struct {
    char name[VARIABLE_NAME_LENGTH];  // Variable name
    int value;                         // Stored value
    int isInitialized;                 // Initialization flag
} Variable;
```

Listing 6: Variable Structure

Operations:

- `addVariable()`: Adds new variable to table
- `setVariableValue()`: Updates variable value
- `lookupVariable()`: Retrieves variable value

5 Expression Evaluation

The parser evaluates arithmetic expressions using recursive descent:

```
int parseExpression() {
    // 1. Parse left operand (number or variable)
    // 2. Check for '+' operator
    // 3. If found, recursively parse right side
    // 4. Return sum or single value
    // Supports: 5, x, 5 + 3, x + 5, x + y + 10
}
```

Listing 7: Expression Parser

Features:

- Right-associative addition
- Supports variable references
- Immediate evaluation during parsing

6 Program Flow

1. **File Input:** Read source file from command line argument
2. **Tokenization:** Convert source code into token stream
3. **Parsing:** Validate syntax against CFG rules
4. **Semantic Analysis:** Check variable declarations and usage
5. **Execution:** Perform arithmetic operations
6. **Output:** Display results via print statements

7 Example Programs

7.1 Example 1: Basic Declaration and Print

Input:

```
int x = 10;
print(x);
```

Output:

```
10
```

7.2 Example 2: Arithmetic Operations

Input:

```
int a = 5;
int b = 10;
int c = a + b + 15;
print(c);
```

Output:

```
30
```

7.3 Example 3: Variable Assignment

Input:

```
int x;
x = 20 + 5;
print(x);
```

Output:

```
25
```

8 Key Components Summary

Component	Function
Tokenizer	Lexical analysis - converts text to tokens
Parser	Syntax analysis - validates CFG rules
Symbol Table	Stores variable information
Error Handler	Detects syntax and semantic errors
Evaluator	Performs arithmetic operations

Table 1: Parser Components

9 Compilation and Execution

Compile:

```
gcc -o parser parser.c
```

Run:

```
./parser input.txt
```